PHP with MySQL — Student Notes

Class 3: PHP Basics — Syntax, Variables, Data Types, Output

1. Recap From Class 2

You now have Apache and MySQL running via XAMPP, and you know PHP code sits inside <?php ... ?> tags and runs on the server. Today's class is the real foundation of the entire course — almost every future class (arrays, functions, OOP, even Laravel) is built directly on top of what you learn here. So we're going deep, not just skimming definitions.

2. What Actually Happens When a PHP File Runs?

Before touching syntax, it's worth understanding the journey a .php file takes, because it explains almost every 'weird' beha vior you'll hit later (blank pages, 'headers already sent' errors, etc.).

- 1. Your browser requests a URL like http://localhost/site/index.php
- 2. Apache sees the .php extension and hands the file to the PHP interpreter (not to itself — Apache doesn't understand PHP)
- 3. The PHP engine (Zend Engine, which you met in Class 1's version history) reads the file top to bottom, LEXES it (breaks it into tokens like $, =, "Hello", ;), then PARSES those tokens into an structure it can execute, and compiles it into opcodes
- 4. The engine executes those opcodes line by line, top to bottom, left to right within a line
- 5. Anything outside <?php ?> tags is streamed to the output buffer as literal text; anything the engine echoes/prints is also streamed to that same buffer
- 6. Once the whole file finishes executing, that output buffer — now a complete HTML document — is sent back to Apache, which sends it to your browser as the HTTP response

> 💡 *This single-pass, top-to-bottom execution model is why order matters enormously in PHP: a variable must be defined BEFORE the line that uses it. PHP does not scan ahead and 'notice' a variable declared further down the file the way some languages hoist declarations.*

3. PHP Syntax Basics — In Depth

Every PHP statement ends with a semicolon (;). This isn't a stylistic preference — it's how the parser (step 3 above) knows where one instruction ends and the next begins. Without it, the tokenizer keeps reading tokens as part of the same statement until it hits something that makes no grammatical sense, and only THEN throws a Parse error — which is why the error line number PHP reports is often one line AFTER your actual mistake.

```
<?php
    echo "This line ends properly";   // semicolon present — correct
    echo "This line is missing one"   // <-- forgot the semicolon!
    echo "This will cause a Parse error";
    // PHP actually reports the error on THIS line, not the line above,
    // because it only realizes something is wrong once it tries to
    // glue the next echo onto the unfinished previous statement.
?>
```

> 💡 *Debugging tip you'll use all semester: when PHP reports 'Parse error: syntax error, unexpected token X on line N', always check line N-1 first, not just line N.*

Opening and Closing Tags — the Full Picture

- <?php ... ?> — the standard, always-works tag. Use this.
- <?= $variable ?> — shorthand for <?php echo $variable; ?>, extremely common inside HTML templates because it's shorter to type repeatedly

- <? ... ?> — 'short open tags', DISABLED by default on modern servers for security/portability reasons. Avoid relying on these; code using them will silently fail to run as PHP on servers where they're off, and instead be sent to the browser as literal visible text.

```
<!-- Inside an HTML template, these two lines do the exact same thing -->
<p><?php echo $studentName; ?></p>
<p><?= $studentName ?></p>   <!-- shorthand, no semicolon needed before ?> -->
```

Why omit the closing ?> in pure-PHP files (files with no HTML at all)? Because PHP treats ANY character after ?> — even a single invisible trailing space or newline your editor added — as literal output to send to the browser. If that file is included before you've sent any real HTML headers (common in login/redirect logic), you get the infamous 'Cannot modify header information — headers already sent' error. Omitting ?> at the very end of a pure-PHP file eliminates this risk entirely.

Comments — Notes for Humans, Ignored by the Engine

```
<?php
    // Single-line comment — everything after // on this line is ignored
    # Also single-line — a less common, Perl/shell-style alternative

    /*
       Multi-line comment.
       Everything between /* and */ is ignored, across as many lines as needed.
    */

    /* /* nested comments do NOT work in PHP */  -- this line would cause an
error,
    // because the FIRST */ closes the comment, and the leftover text after it
    // is then treated as real code and breaks the script.
?>
```

💡 *Comments are removed during the lexing step (Section 2) — they cost zero execution time and are never sent to the browser, so there's no performance reason to avoid writing them generously. Industry convention: comment the WHY behind a decision, not the WHAT the code obviously already says.*

4. Your First PHP Script — "Hello World", Explained Line by Line

```
<!DOCTYPE html>
<html>
<body>

<?php
    echo "Hello, World!";
?>

</body>
</html>
```

Walk through exactly what leaves the server for the browser: everything from <!DOCTYPE html> down to <body> is plain text — PHP doesn't touch it, it flows straight into the output buffer. Then the engine hits <?php, switches into 'execution mode', runs echo "Hello, World!";, which appends the literal text Hello, World! to that same output buffer, then hits ?> and switches back to 'plain text mode' for </body></html>. The browser never knows PHP was involved at all — if you View Page Source, you will see exactly:

```
<!DOCTYPE html>
<html>
<body>
```

```
Hello, World!

</body>
</html>
```

Save the first version as hello.php inside a folder under htdocs, then visit it via http://localhost/yourfolder/hello.php. Double-clicking the file instead of going through Apache will show you the raw <?php echo ...; ?> as text, because no PHP engine ever touched it — this is one of the most common Week-1 confusions.

5. Variables — A Much Deeper Look

> 💡 *A variable is a named reference to a value stored in memory. In PHP, every variable name starts with a dollar sign ($), and — unlike C or Java — you never declare its type. The type is a property of the VALUE currently inside the variable, not of the variable name itself.*

Real-World Analogy: Labeled Storage Boxes (Extended)

A labeled box ($bookBox) can hold a novel today and a pile of old newspapers next month — the label never changes, only the contents. Two important extensions to this analogy that matter in PHP specifically:

- If you write $secondBox = $bookBox; PHP does NOT make $secondBox point to the same physical box — for simple values (strings, numbers, booleans) it photocopies the contents into a brand-new box. Changing $secondBox afterward never affects $bookBox. This is called 'assignment by value.'
- You CAN explicitly make two labels point to the exact same box using &, called a reference: $secondBox = &$bookBox; — now both labels open the same physical box, and changing one changes what the other sees. You'll use this deliberately in Class 10 (call by reference in functions).

```
<?php
    $bookBox = "Notes";
    $secondBox = $bookBox;      // COPY — a new, independent box
    $secondBox = "Novels";
    echo $bookBox;              // Still 'Notes' — untouched

    $thirdBox = &$bookBox;      // REFERENCE — same physical box, two labels
    $thirdBox = "Newspapers";
    echo $bookBox;              // Now 'Newspapers' too — same box!
?>
```

Naming Rules (Frequently Tested)

- Must start with a letter or underscore — never a digit ($1name is invalid, $name1 is fine, $_temp is valid)
- Can only contain letters, numbers, and underscores — no spaces, hyphens, or special characters
- Case-sensitive — $Name, $NAME, and $name are three completely different variables. This trips up beginners constantly and is worth deliberately breaking once in lab so you recognize the bug pattern later.

Variable Variables — An Advanced Curiosity

PHP allows a variable's NAME to itself be stored in another variable, using a double dollar sign. You won't use this often, but interviewers sometimes ask about it to test how well you understand PHP's variable model:

```
<?php
    $fieldName = "email";
    $$fieldName = "student@example.com";   // creates a NEW variable called
$email
    echo $email;                            // Outputs: student@example.com
?>
```

6. Variable Scope — Every Type, In Depth

Scope answers one question: from which parts of your code can a given variable be seen and used? Getting this wrong is one of the single most common sources of 'my variable is undefined' or 'my function isn't updating the value I expected' bugs.

Scope	Meaning	Real-world parallel
Local	Exists only inside the function it's declared in; destroyed when the function finishes running	A note you scribble inside your own private notebook — no one outside your room sees it, and it's thrown away once you close the book
Global	Declared outside any function; NOT automatically visible inside functions — you must explicitly say 'global $varName;' inside the function to access it	An announcement on the college notice board — it exists for everyone, but you must physically walk to the board and read it; it doesn't follow you into a locked exam room unless you bring a copy
Static	A local variable that keeps its value between successive calls to the same function, instead of resetting each time	A shop's token-number counter — it does NOT reset to zero for every new customer; it remembers the last number given out
Superglobal	Built-in arrays automatically available EVERYWHERE — inside every function, every file, with no 'global' keyword needed	An emergency siren — reaches every room in the building instantly, no one has to walk anywhere to hear it

Local vs Global — Full Worked Example

```
<?php
    $balance = 1000;              // GLOBAL variable

    function showBalanceWrong() {
        echo $balance;             // ERROR / Undefined — function can't see the
outer $balance
    }

    function showBalanceRight() {
        global $balance;           // explicitly PULL IN the outer variable
        echo $balance;              // Now works — outputs 1000
    }

    function withdrawWrong($amount) {
        $balance = $balance - $amount;   // creates a NEW local $balance, outer
one untouched
    }

    function withdrawRight($amount) {
        global $balance;
        $balance = $balance - $amount;   // now correctly modifies the outer
variable
    }

    withdrawWrong(200);
    echo $balance;   // Still 1000 — the 'wrong' function never touched the real
one

    withdrawRight(200);
```

```
    echo $balance;   // Now 800 — correctly updated
?>
```

💡 *Why does PHP design it this way instead of letting every function freely see every outer variable? Predictability and safety — if any of your 40 functions could silently modify any global variable without saying so, a bug in one function could corrupt data used by a completely unrelated part of a 5,000-line project, and you'd have no way to know where to even start looking.*

Static — Full Worked Example

```
<?php
    function generateTokenNumber() {
        static $counter = 0;   // initialized ONCE, on the very first call only
        $counter++;
        return $counter;
    }

    echo generateTokenNumber();   // 1
    echo generateTokenNumber();   // 2
    echo generateTokenNumber();   // 3 — remembers previous calls, unlike a
normal local variable
?>
```

Superglobals — The Full List You'll Use All Semester

Superglobal	Holds	When you'll use it
$_GET	Data sent via the URL query string (?name=value)	Class 15/17 — reading search filters, links with parameters
$_POST	Data submitted via an HTML form's POST method	Class 15 — form submissions like registration, login
$_REQUEST	A merge of $_GET, $_POST, and $_COOKIE	Class 17 — rarely recommended in real projects since it's ambiguous about the data's source
$_SESSION	Data that persists across multiple page requests for one logged-in user	Class 18 — login systems, shopping carts
$_COOKIE	Small data stored in the user's browser and resent on every request	Class 18 — 'remember me' functionality
$_SERVER	Information about the server and current request (URL, method, headers)	Class 17 — checking REQUEST_METHOD, building URLs dynamically
$_FILES	Information about files uploaded via a form	Class 17 — profile picture / document upload

💡 *You'll formally use most of these from Class 15 onward, but recognizing the full family now means when $_POST first appears in three weeks, it won't feel like a brand-new unrelated concept — it's the same 'superglobal' idea you learned today, just a different named box.*

7. Constants — Deeper Rules

A constant is a name bound permanently to a value for the rest of the script's execution — no variable, function, or accident anywhere in your code can change it afterward. PHP gives you two ways to define one, and they behave slightly differently:

	define()	const
Where can it be used?	Anywhere, including inside if-blocks and functions — evaluated at runtime	Only at the top level of a file or inside a class — evaluated at compile time
Can the value be an expression?	Yes, any expression (e.g., a function's return value)	Only simple constant expressions (no function calls)
Common use	Configuration values decided conditionally	Fixed values known in advance, and CLASS constants (Class 30)

```php
<?php
    define("COLLEGE_NAME", "Techno Main Salt Lake");
    const TAX_RATE = 0.18;

    echo COLLEGE_NAME;   // No $ sign — this is what visually distinguishes
    echo TAX_RATE;       // a constant from a variable at a glance

    // COLLEGE_NAME = "Something Else";   // This is not even valid syntax —
                                          // constants can never be reassigned
?>
```

Real-world parallel: a constant is like your official date of birth on a government ID — referred to constantly throughout many processes, but no process is ever allowed to silently edit it.

Magic Constants

PHP has nine predefined constants that change value depending on where they are used, also called the "magic constants". The magic constants provide information about the current state of the script, such as the file name, directory name, line number, function name, class name, and more.
The magic constants always start and end with double underscores (__), except for the ClassName::class constant.
The magic constants are not case-sensitive, meaning __LINE__ returns the same as __line__.
The following table lists the magic constants, with descriptions and examples:

Magic Constant	Gives you	Typical real use
__CLASS__	If used inside a class, the class name is returned.	Inside a class method, logging or error messages that need to say "this problem happened in class X" without hardcoding the class name — handy when a class gets renamed later or is extended by a child class.
__DIR__	The directory of the file.	Building reliable file paths for include/require (e.g., require __DIR__ . '/config.php';) so includes work correctly no matter which script originally called the file — a very common real bug fix for "file not found" errors caused by relative paths.
__FILE__	The full filesystem path of the current file	Building reliable include/require paths, or logging which file caused an issue
__FUNCTION__	The name of the function currently executing	Generic logging code that reports its own caller without hardcoding the name
__LINE__	The current line number in the file	Pinpointing exactly where a custom error/log message was triggered
__METHOD__	If used inside a function that belongs to a class, both class and function name is returned.	Debug/audit logging inside class methods — e.g., recording "User::login was called at ..." automatically, useful for tracing exactly which class-and-method combination triggered an action in a large OOP project.

Magic Constant	Gives you	Typical real use
__NAMESPACE__	If used inside a namespace, the name of the namespace is returned.	Used in larger, namespaced applications (common once you reach Laravel) to dynamically build fully-qualified class names or debug messages without hardcoding the namespace string.
__TRAIT__	If used inside a trait, the trait name is returned.	Debugging/logging inside a trait itself — since the same trait can be reused across many unrelated classes (Class 32), this tells you which trait's code actually ran, separate from __CLASS__, which would report the *using* class instead.
ClassName::class	Returns the name of the specified class and the name of the namespace, if any.	The modern, refactor-safe way to get a class's fully-qualified name as a string — e.g., for type checks (instanceof), Laravel service bindings, or exception handling (catch (SomeException::class ...)), instead of writing the class name as a fragile plain string that breaks silently if the class is renamed.

```
<?php
    function processPayment() {
        echo "Error occurred in " . __FUNCTION__ . " at line " . __LINE__;
        // Error occurred in processPayment at line 3
    }
?>
```

8. Data Types — A Genuine Deep Dive

PHP is loosely typed (also called dynamically typed): you never declare a variable's type upfront, and the SAME variable name can hold a string in one line and an integer three lines later. Contrast this with a strictly typed language like Java, where 'int age = 20;' permanently locks that variable to integers only. PHP instead looks at the VALUE and decides the type on the spot, every single time.

Category	Type	Example	Real-world example
Scalar	String	"Hello", 'Kolkata'	A name, an address, any text
Scalar	Integer	20, -5, 100	Age, roll number, quantity
Scalar	Float (double)	8.75, 3.14	CGPA, price, temperature
Scalar	Boolean	true, false	Is the user logged in? Yes/No
Compound	Array	["apple", "banana"]	A shopping list, a class attendance sheet (full detail in Week 3)
Compound	Object	an instance of a class	A specific student record with its own name/roll/marks (full detail from Class 27)
Special	NULL	null	'No value at all' — an empty mailbox, not even a blank letter inside it

Strings, In Real Depth

PHP has FOUR ways to write string literals, and the choice genuinely changes behavior, not just style:

```
<?php
    $name = "Priya";

    // 1. Single quotes – everything is LITERAL. Variables do NOT get replaced.
```

```
    echo 'Hello $name';        // Outputs literally: Hello $name

    // 2. Double quotes — PHP PARSES the string, replacing variables and escape
codes
    echo "Hello $name";        // Outputs: Hello Priya
    echo "Line1\nLine2";       // \n becomes an actual newline

    // 3. Heredoc — like double quotes, but for large multi-line blocks
    echo <<<EOT
    Dear $name,
    Your admission to Techno Main Salt Lake is confirmed.
    EOT;

    // 4. Nowdoc — like single quotes, but for large multi-line LITERAL blocks
    echo <<<'EOT'
    This $name will NOT be replaced — printed exactly as written.
    EOT;
?>
```

Interview-relevant, and a genuine performance point: single quotes are marginally faster than double quotes for strings with NO variables inside them, because PHP skips the parsing step entirely. On a small script this is irrelevant, but it's a real (if minor) reason experienced developers default to single quotes unless interpolation is actually needed.

Integers — Limits You Should Know

An integer in PHP is typically stored using 64 bits on modern systems, giving a maximum value of 9,223,372,036,854,775,807 (accessible via the constant PHP_INT_MAX). Cross that limit, and PHP doesn't error — it silently converts the number to a float instead, which can quietly lose precision for very large numbers. This matters in real systems doing financial or scientific calculations at scale.

Floats — A Trap Every Language Shares

```
<?php
    $result = 0.1 + 0.2;
    echo $result;                    // Prints 0.3 on screen (rounded display)
    var_dump($result == 0.3);        // Often FALSE — the real stored value
                                       // is something like 0.30000000000000004
?>
```

This isn't a PHP bug — it's how ALL languages store floating-point numbers in binary, because certain decimal fractions (like 0.1) can't be represented exactly in binary, the same way 1/3 can't be written exactly as a finite decimal. Real-world consequence: NEVER compare two floats with == for equality in financial calculations; either round both sides to a fixed number of decimals first, or store money as integers (paise instead of rupees) and convert only for display.

Booleans — The Complete Falsy List

Only these values are considered false when converted to boolean — genuinely everything else in PHP is true, including strings that look 'negative' in plain English:

- false itself
- The integer 0 and float 0.0
- An empty string "" and the string "0" specifically (but NOT "0.0" or "false" — those are truthy!)
- An empty array []
- null

```
<?php
    var_dump((bool) "0");       // false  – the ONE special falsy string
    var_dump((bool) "0.0");     // true   – looks similar but is NOT special-
cased
    var_dump((bool) "false");   // true   – the ENGLISH WORD is just non-empty
text
    var_dump((bool) " ");       // true   – a single space is non-empty
    var_dump((bool) []);         // false  – empty array
    var_dump((bool) [0]);        // true   – array containing one element (even a
falsy one!)
?>
```

This exact list is one of the most frequently tested PHP interview questions, because it directly explains dozens of real production bugs — e.g., checking `if ($_POST['quantity'])` silently fails when a user legitimately submits the string "0".

9. Type Juggling vs Type Casting — The Full Mechanics

Type juggling = PHP automatically converting a value's type behind the scenes, based on the operator being used. Type casting = YOU explicitly forcing a value into a chosen type using (int), (float), (string), (bool), or (array).

```
<?php
    // Type Juggling examples — PHP decides automatically
    $result = "5" + 3;        // '+' expects numbers, so "5" is juggled to int 5
    echo $result;              // Outputs: 8

    $result2 = "5" . 3;       // '.' is the STRING concatenation operator (Class
4),
    echo $result2;             // so both sides are juggled to strings: Outputs
53

    $result3 = "5 apples" + 3;
    echo $result3;             // Outputs: 8, with a WARNING — PHP reads the
leading
                               // numeric part '5' and discards ' apples'

    // Type Casting examples — you decide explicitly
    $price = "499.50";
    $numericPrice = (float) $price;
    echo $numericPrice + 10;   // Outputs: 509.5

    $count = (int) "12abc";   // 12 — leading digits kept, rest discarded
    $flag  = (bool) "0";      // false — see the falsy list above
?>
```

Checking and Changing Types Explicitly

```
<?php
    $value = "42";

    echo gettype($value);        // string
    var_dump(is_string($value)); // true
    var_dump(is_numeric($value));// true — it LOOKS numeric even though it's
stored as a string
```

```
    settype($value, "integer");  // permanently converts $value itself, not a
copy
    echo gettype($value);        // integer
?>
```

is_numeric() vs is_int()/is_string() is a genuinely important distinction: a value coming from an HTML form (Class 15 onward) always arrives as a STRING, even if the user typed '25'. is_numeric('25') is true, but is_int('25') is false. Confusing these two checks is a very common beginner validation bug.

10. Output & Debugging Functions — In Depth

PHP gives you several ways to display data, each suited to a different real situation — similar to how a doctor might give a one-line verbal update (echo) versus a full printed lab report (var_dump).

Function	Use for	Example output
echo	Quick output; can take multiple comma-separated values; technically a language construct, not a function, so it has no return value	Hello, World!
print()	Same purpose as echo, but only takes ONE argument and always returns 1 — occasionally used inside expressions because of that return value	Hello, World!
printf()	Formatted output using placeholders: %s string, %d integer, %.2f float rounded to 2 decimals	CGPA: 8.75
sprintf()	Identical formatting to printf(), but RETURNS the formatted string instead of printing it — used when you need to store or reuse the text	(stored in a variable, not shown)
print_r()	Human-readable structure of arrays/objects — the everyday debugging workhorse for lists	Array ([0] => apple)
var_dump()	Full detail: shows the TYPE and VALUE of anything, including nested arrays/objects — the most precise debugging tool	string(5) "apple"
var_export()	Like var_dump, but outputs valid, re-runnable PHP code representing the value	array (\n 0 => 'apple',\n)

```
<?php
    $name = "Rahul";
    $cgpa = 8.5;

    echo "Name: $name, CGPA: $cgpa";                // Simple, quick, no
formatting control
    printf("Name: %s, CGPA: %.2f\n", $name, $cgpa); // Name: Rahul, CGPA: 8.50 —
2 decimals guaranteed

    $message = sprintf("Welcome, %s!", $name);      // Stored in $message, NOT
printed yet
    echo $message;                                    // Now we print it
ourselves
```

Function	Use for	Example output

```
    $marks = [78, 85, 91];
    print_r($marks);
    // Array ( [0] => 78 [1] => 85 [2] => 91 )

    var_dump($cgpa);
    // float(8.5)   <- shows the TYPE too, which print_r never does

    var_dump("8.5");
    // string(3) "8.5"  <- same-looking value, completely different type — this
is
    //                     EXACTLY the kind of bug var_dump exists to catch
?>
```

Real industry habit: when a form value, a database result, or an API response 'isn't working as expected' in an if-condition, the very first debugging step most PHP developers reach for is var_dump ($value); die; right above the problem line — it immediately reveals whether the value is the string "0" instead of the integer 0, whether it's null instead of an empty string, and so on. You will use this constantly for the rest of the course.

A Quick Note on isset(), empty(), and is_null()

These three are technically 'output/inspection' tools too, and get confused with each other constantly:

Function	Returns true when...	Real use case
isset($v)	$v exists AND is not null	Checking whether a form field or array key was ever set at all
empty($v)	$v doesn't exist, OR is one of the falsy values from Section 8	Checking whether a user left a required field blank
is_null($v)	$v exists and its value is exactly null	Distinguishing 'never set' from 'deliberately set to nothing'

11. Live Demo (What We'll Do In Lab)

- Live-code Hello World and view the actual HTML source the browser receives, to prove PHP never reaches the client
- Declare one variable of every scalar type, then run echo, print_r, and var_dump on all of them side by side to see exactly how each function differs
- Deliberately trigger and read a real Parse error (missing semicolon) and a real Warning (undefined variable), to build comfort reading PHP's error messages

12. Practice Problems (In-Class)

- Declare one variable of each scalar type and print each with var_dump() — predict the output on paper first, then check
- Reproduce the local-vs-global scope bug from Section 6, then fix it using the 'global' keyword
- Write three expressions using "0", "0.0", and "false" cast to boolean, predict each result, then verify with var_dump()

13. Homework

Write a script that stores your name, age, and CGPA in variables, then prints a formatted sentence using printf() — e.g., "My name is Rahul, I am 20 years old, and my CGPA is 8.50." Additionally, write one line demonstrating a type-juggling surprise (like the "5 apples" + 3 example) and explain in a comment why PHP produced that output.

14. Interview Questions from This Class

Question	Answer
Is PHP a strictly typed or loosely typed language?	Loosely (dynamically) typed — a variable's type is determined by its current value at runtime, not declared upfront, and can change if a new value of a different type is assigned to it.
What is the difference between echo and print?	echo can output multiple comma-separated values and has no return value; print accepts exactly one argument and always returns 1, which occasionally makes it usable inside larger expressions.
What's the difference between print_r(), var_dump(), and var_export()?	print_r() gives a readable structure of arrays/objects but hides scalar types; var_dump() shows both type and value for anything, including nested data; var_export() outputs valid PHP code representing the value, useful for generating code or config.
What is the difference between == and === (preview)?	== compares values after type juggling (loose comparison); === compares both value AND type with no conversion (strict comparison) — this is covered fully in Class 4, but understanding today's type system is the prerequisite for it.
What is variable scope, and name PHP's scope types.	Scope defines where a variable can be accessed: Local (inside its own function only), Global (outside functions; needs the 'global' keyword to be reached inside one), Static (local, but retains its value between calls), and Superglobal (built-in arrays like $_POST, accessible everywhere automatically).
What are PHP's falsy values?	false, 0, 0.0, "" (empty string), "0" (the specific string zero), an empty array [], and null. Every other value, including the strings "0.0" and "false", is truthy.
What's the difference between isset(), empty(), and is_null()?	isset() is true if a variable exists and isn't null; empty() is true if a variable doesn't exist or holds a falsy value; is_null() is true only if the variable exists and its value is specifically null.
Why shouldn't you compare floating-point numbers with == for equality?	Because most decimal fractions cannot be represented exactly in binary floating-point, so two mathematically equal-looking calculations (like 0.1 + 0.2 vs 0.3) can differ by a tiny margin internally — comparisons should use rounding or a small tolerance instead.

15. FAQs

Question	Answer
Why did my script show nothing, and instead I saw an error about 'unexpected token'?	This is almost always a missing semicolon on the line ABOVE the one PHP reports, or a mismatched quote — because PHP only realizes something is wrong once it tries to combine the broken statement with the next line.
Do I need to write something like var $name; before using a variable, like other languages?	No — PHP has no variable declaration keyword. Writing $name = "value"; both creates the variable and assigns it in one step; there is nothing to declare in advance.
What's the real difference between single quotes ' ' and double quotes " " for strings?	Double quotes are PARSED — variables and escape sequences like \n inside them are replaced with their real values; single quotes are LITERAL — everything inside is printed exactly as typed except \' and \\.
Why does PHP let me change a variable's type mid-script? Isn't that dangerous?	It's a deliberate trade-off: loose typing makes small scripts and rapid prototyping faster to write, at the cost of type-related bugs being caught later (at runtime) rather than earlier (at compile time). PHP 7+ added

Question	Answer
	optional strict typing (declare(strict_types=1), covered in Class 12) specifically so larger, more serious projects can opt into the safety of strict types where it matters.
Is there a real performance difference between echo and print, or single vs double quotes?	The differences are measurable in benchmarks but are genuinely tiny for typical student/college-project scale code — they matter far more at the scale of a high-traffic production site processing millions of requests. Learn the correct behavioral differences first; treat the performance angle as a 'nice to know' fact, not a rule to obsess over yet.

— End of Class 3 Notes —